UI EXPERIMENT 3

1.Command Line Interface

- A Command Line Interface (CLI) allows the user to interact with the system by typing commands and inputs through the keyboard. It is simple and fast because the user directly enters data without using buttons or graphics.
- CLI programs require less memory and system resources compared to graphical applications. In this experiment, the user enters subject marks through the terminal, and the system calculates and displays the grade using text-based interaction.

IMPLEMENTATION:

```
# bmi_cli.py


def calculate_bmi(weight, height):
    bmi = weight / (height ** 2)
    return bmi


def bmi_category(bmi):
    if bmi < 18.5:
        return "Underweight"
    elif 18.5 <= bmi < 24.9:
```

```
        return "Normal weight"
    elif 25 <= bmi < 29.9:
        return "Overweight"
    else:
        return "Obese"

def main():
    print("BMI Calculator")
    weight = float(input("Enter your weight (kg): "))
    height = float(input("Enter your height (m): "))

    bmi = calculate_bmi(weight, height)
    category = bmi_category(bmi)

    print("\nResult")
    print(f"Your BMI: {bmi:.2f}")
    print(f"Category: {category}")

if __name__ == "__main__":
    main()
```

OUTPUT:

```
PROBLEMS    OUEPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C.\Users\deshna\Desktop\UI_Project> pytnon bmi_cli.py
>>
BMI Calculator
Enter your weight (kg): 72
Enter your height (m): 1.75

Result
Your BMI: 23.51
Category: Normal wweight
PS C.\Users\deshna\Desktop\UI_Project>
```

2.GRAPHICAL USER INTERFACE

- A Graphical User Interface (GUI) allows users to interact with the system using visual elements like windows, buttons, and text boxes instead of typing commands. It is user-friendly and easy to understand because users can click buttons and enter data in input fields.
- GUI provides a better visual experience and is suitable for beginners who may not know command-line instructions. In this experiment, the user enters subject marks in text boxes, clicks a button, and the system displays the total, average, and grade in a window.

IMPLEMENTATION:

```
import tkinter as tk

from tkinter import messagebox

import requests
```

```
from bs4 import BeautifulSoup

def scrape_data():
    url = entry.get()
    try:
        response = requests.get(url, timeout=5)
        soup = BeautifulSoup(response.text, 'html.parser')

        # Find the first H1 tag
        heading = soup.find('h1')
        result_text = heading.text.strip() if heading else "No
H1 tag found."

        label_result.config(text=f"Result:        {result_text}",
fg="green")
    except Exception as e:
        messagebox.showerror("Error", "Check the URL or
your connection")

# GUI Setup
root = tk.Tk()
root.title("Quick Scraper")
```

```
root.geometry("400x250")

tk.Label(root, text="Enter Website URL:", pady=10).pack()
entry = tk.Entry(root, width=40)
entry.insert(0, "https://www.example.com")
entry.pack()

tk.Button(root, text="Scrape Heading", command=scrape_data,
 bg="blue", fg="white").pack(pady=20)

label_result = tk.Label(root, text="Result: (Waiting...)", wraplength=350)
label_result.pack()

root.mainloop()
```

OUTPUT:

3.VOICE USER INTERFACE

- A Voice User Interface (VUI) allows users to interact with the system using voice commands instead of typing or clicking. It makes interaction hands-free and more natural, as users can simply speak to give input.
- VUI uses speech recognition technology to convert spoken words into text and process the request. In this experiment, the user speaks the marks through a microphone, and the system recognizes the voice input, calculates the grade, and displays the result.

IMPLEMENTATION:

```
import speech_recognition as sr

import pyttsx3

import time


# Initialize the "Voice" (Text-to-Speech)

engine = pyttsx3.init()

engine.setProperty('rate', 175) # Speed of speech
```

```
def speak(text):
    print(f"🤖 System: {text}")
    engine.say(text)
    engine.runAndWait()

def listen():
    recorder = sr.Recognizer()
    with sr.Microphone() as source:
        print("🎤 Listening...")
        recorder.pause_threshold = 1
        audio = recorder.listen(source)

    try:
        print("🌀 Recognizing...")
        query = recorder.recognize_google(audio, language='en-in')
        print(f"🧑 You said: {query}\n")
        return query
    except Exception:
        speak("I didn't catch that. Could you say it again?")
        return "None"
```

```
# --- Main Program ---
speak("Voice User Interface Activated. Welcome!")

while True:
    speak("Please say your marks for three subjects separated by space.")
    voice_input = listen()

    if voice_input == "None":
        continue

    try:
        # Convert the spoken words into a list of integers
        marks = list(map(int, voice_input.split()))

        if len(marks) != 3:
            speak("Please provide exactly three numbers.")
            continue

        average = sum(marks) / 3
```

```
        grade = "A" if average >= 90 else "B" if average >= 75 else "C" if average >= 60 else "F"


        result_msg = f"Your average is {average:.1f}. Your grade is {grade}."
        speak(result_msg)
        break # Exit after success


    except ValueError:
        speak("I heard your voice, but those didn't sound like numbers. Try again.")


speak("Calculation complete. Goodbye!")
```